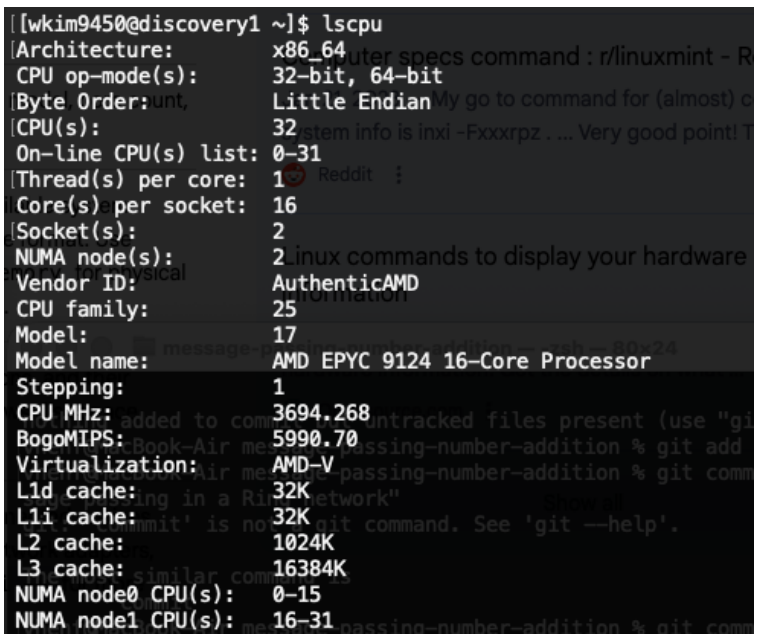
EE-451: Parallel and Distributed Computation

Wooyoung Kim(2717106812) phw5

April 20th 2026

Hardware Specification (CARC)



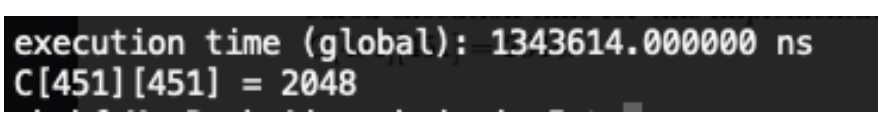
```
[wkim9450@discovery1 ~]$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                32
On-line CPU(s) list:   0-31
Thread(s) per core:    1
Core(s) per socket:    16
Socket(s):             2
NUMA node(s):          2
Vendor ID:             AuthenticAMD
CPU family:            25
Model:                 17
Model name:            AMD EPYC 9124 16-Core Processor
Stepping:              1
CPU MHz:               3694.268
BogoMIPS:              5990.70
Virtualization:        AMD-V
L1d cache:             32K
L1i cache:             32K
L2 cache:              1024K
L3 cache:              16384K
NUMA node0 CPU(s):     0-15
NUMA node1 CPU(s):     16-31
```

Figure 1: Hardware specification for homework

1. Matrix Multiplication: Memory Optimization

Problem 1.1: Global Memory Implementation (p1_memory.cu)

The unoptimized matrix multiplication utilizes global memory for all read and write operations. The kernel accesses the matrices directly without leveraging the shared memory cache explicitly. The measured execution time for this implementation is **1343614 ns**(1.34 ms), producing the expected output value $C[451][451] = 2048$.



```
execution time (global): 1343614.000000 ns
C[451][451] = 2048
```

Figure 2: 1.1 Output

Problem 1.2: Shared Memory Tiling (p2_memory.cu)

The optimized version implements block matrix multiplication using shared memory tiling. By loading tiles of the matrices into the much faster shared memory, this approach significantly reduces global memory traffic. The execution time drops to **728655 ns** (0.73 ms). This represents a nearly 1.84 times speedup over the global memory implementation. The output remains consistent at $C[451][451] = 2048$.

```
execution time (shared): 728655.000000 ns
C[451][451] = 2048
```

Figure 3: 1.2 Output

2. Matrix Multiplication: Asynchronous Streams

Problem 2.1: Sequential Issue Loop (p1_async.cu)

This approach overlaps memory copies (Host-to-Device and Device-to-Host) with kernel execution using asynchronous CUDA streams. Commands are issued in a single loop sequentially. `cudaMemcpyAsync(H2D)` → `Kernel Launch` → `cudaMemcpyAsync(D2H)`.

The execution times were:

- **1 Stream:** 6.856 ms
- **4 Streams:** 6.332 ms
- **16 Streams:** 6.127 ms

The target output value correctly matched the expectations: $C[451][451] = 208282624$.

```
Streams:  1, Time:    6.856 ms, C[451][451] = 208282624
Streams:  4, Time:    6.332 ms, C[451][451] = 208282624
Streams: 16, Time:    6.127 ms, C[451][451] = 208282624
```

Figure 4: 2.1 Output

Problem 2.2: Parallel Issue Loops (p2_async.cu)

In the parallel issue approach, commands are dispatched in three separate loops: all H2D copies, followed by all kernel launches, and finally all D2H copies. The measured execution times were:

- **1 Stream:** 6.790 ms
- **4 Streams:** 6.292 ms
- **16 Streams:** 5.968 ms

The target output value $C[451][451] = 208282624$ was verified.

```
Streams:  1, Time:    6.790 ms, C[451][451] = 208282624
Streams:  4, Time:    6.292 ms, C[451][451] = 208282624
Streams: 16, Time:    5.968 ms, C[451][451] = 208282624
```

Figure 5: 2.2 Output

3. Performance Discussion

Execution Time Comparison Table

Table 1 summarizes the performance characteristics of all tested configurations.

Category	Implementation	Streams	Time (ms)
Memory Optimization	P1.1: Global Memory	-	1.344
	P1.2: Shared Memory Tiling	-	0.729
Asynchronous Streams (Seq)	P2.1: Sequential Issue Loop	1	6.856
	P2.1: Sequential Issue Loop	4	6.332
	P2.1: Sequential Issue Loop	16	6.127
Asynchronous Streams (Par)	P2.2: Parallel Issue Loops	1	6.790
	P2.2: Parallel Issue Loops	4	6.292
	P2.2: Parallel Issue Loops	16	5.968

Table 1: Performance comparison of all matrix multiplication implementations.

Impact of the Number of Streams

As seen in the results for both asynchronous implementations, increasing the number of streams from 1 to 4, and then to 16, consistently reduces the overall execution time. With 1 stream, the operations are essentially serialized, offering no overlap between data transfers and computation. As the number of streams increases, the CUDA runtime overlaps the H2D memory transfers of one stream with the kernel computation of another stream, and the D2H transfers of yet another. This concurrency hides the latency of PCIe memory transfers and maximizes the utilization of the GPU's copy and compute engines, leading to lower total execution times.

Sequential vs. Parallel Issue Loops

Comparing Problem 2.1 and Problem 2.2, a speedup is observed when utilizing the parallel issue loop, particularly at higher stream counts (5.968 ms vs. 6.127 ms at 16 streams). The sequential issue loop (H2D $\rightarrow$ Kernel $\rightarrow$ D2H inside a single loop) can sometimes create false dependencies in the hardware queues depending on the GPU architecture, limiting the runtime's ability to properly schedule and overlap tasks. By separating the operations into three distinct loops (all H2D, all Kernels, all D2H), we allow the hardware to pack the work queues much more efficiently, resulting in better overlap and faster overall execution.